

Q. Spring framework?

Ans: Open source development framework for Enterprise Java.

It helps Java programmer in development. It provides dependency Injection and IOC container, Spring MVC flow and several useful API for Java programmer.

It is mainly used in developing a Java Enterprise application.

1. With Spring we can develop large-scale complex Java applications very easily.

2. It is based on good design patterns like Dependency Injection, Aspect oriented programming for developing extensible feature rich software.

3. Lightweight Framework

4. Container: Spring provides the basic container that creates and manages the life cycle of application objects like Plain old Java objects (POJO). It also stores the configuration files of application objects to be created.

5. Dependency Injection (DI): Spring provided loose coupling in application by Dependency Injection. It uses Inversion of Control technique by which objects specify their dependencies to Spring container instead of creating new objects themselves.

6. Aspect Oriented Programming (AOP): Spring framework promotes and provides support for Aspect oriented programming in Java. This helps in separating application business logic from system services that are common across all the business logic. E.g. Logging can be a cross cutting concern in an Application.

7.Transaction Management: Spring provides a framework for transaction management. So a developer does not have to implement it from scratch. Spring Transaction Management is so powerful that we can scale it from one local transaction to global transactions in a cluster.

8.MVC Framework: For Web applications, Spring provides MVC framework. This framework is based on MVC design pattern and has better features compared to other web frameworks. Exception Handling: Spring also gives support for a common API to handle exceptions in various technologies like- Hibernate, JDBC etc.

Q. Modules in Core Container of Spring framework?

Ans:Modules in Core Container are:

- 1.Core module
- 2.Bean module
- 3.Context module
- 4.Spring Expression Language module

Q. Does Spring singleton beans are thread-safe ?

Ans: No, Spring singleton beans are not thread-safe.

Q. What are the modules in Web layer of Spring framework?

Ans:Modules in Web Layer of Spring framework are:

Web module: This provides provides basic web-oriented integration features, such as multipart file-upload functionality, the initialization of the IoC container using servlet listeners and a web-oriented application context. It also contains the web related parts of Spring's remoting support.

Servlet module: Support for Servlet Listeners.

WebSocket module: Support for Web Socket style messaging.

Portlet module: the MVC implementation to be used in a portlet environment and mirrors what is provided in the Web-Servlet module.

Q. Main use of Core Container module in Spring framework?

Ans: It is the core of Spring framework. It gives the basic functionality of the Spring. All the parts of Spring Framework are built on top of Core Container.

Its main use is to provide Dependency Injection (DI) and Inversion of control (IOC) features.

Q.What is the use of BeanFactory in Spring?

Ans:BeanFactory is the main class that helps in implementing Inversion of Control pattern in Spring. It is based on the factory design pattern. It separates the configuration and dependencies of an

application from the rest of application code. Implementations of BeanFactory like XmlBeanFactory class are used by applications built with Spring.

**

The most commonly used simple BeanFactory implementation is `org.springframework.beans.factory.xml.XmlBeanFactory`.

Q. What is XmlBeanFactory in Spring framework?

Ans: This factory loads its beans based on the definitions mentioned in an XML file. Spring container reads bean configuration metadata from an XML file and creates a fully configured application with the help of XmlBeanFactory class.

Q. What are the uses of AOP module in Spring framework?

Ans: AOP module is also known as Aspect Oriented Programming module. Its uses are: Development of aspects in a Spring based application Provides interoperability between Spring and other AOP frameworks Supports metadata programming to Spring

Q. Benefits of JDBC abstraction layer module in Spring framework?

Ans: Spring provides JDBC abstraction layer module. Main benefits of this module are: Helps in keeping the database code clean and simple. Prevents problems that result from a failure to close database resources. Provides a layer of useful exceptions on top of the error messages given by different database servers. Based on Spring's AOP module Provides transaction management services for objects in a Spring application.

Q. How does Spring support Object Relational Mapping (ORM) integration?

Ans: Spring supports Object Relational Mapping (ORM) by providing ORM Module. This module helps in integrating with popular ORM framework like Hibernate, JDO, and iBATIS SQL Maps etc. Transaction Management module of Spring framework supports all of these ORM frameworks as well as JDBC.

Q. Main uses of Spring MVC module?

Ans: Spring-webmvc module is also known as Web-servlet module. It is based on Web Model View Controller pattern.

Main uses of this module are: Integration of Spring with other MVC frameworks Supports IoC to provide clean separation of controller logic from business objects Provides clean separation between domain model code and web forms Allows developers to declaratively bind request parameters to business objects.

Q. How does Web module work in Spring?

Ans: Spring provides support for developing web application by using Web module. This module is built on application context module that provides context for web-based applications. This module also supports web-oriented integration features like- transparently handling multipart requests for uploading files, programmatically binding request parameters to business objects etc. This module also supports integration with popular web frameworks like Jakarta Struts, JSF, and Tapestry etc.

Q. What is the purpose of Spring configuration file?

Ans: Spring application can be configured by an XML file. This file contains information of classes and how these classes are configured and introduced to each other.

Spring IoC container uses some kind of configuration metadata. This configuration metadata represents how an application developer tells the Spring container to instantiate, configure, and assemble the objects in your application. This configuration metadata is stored in Spring configuration file.

The other ways of specifying configuration metadata are Java based configuration and Annotation based configuration.

Q. What is the purpose of Spring IoC container?

Ans: It is responsible for Creating the objects Configuring the objects Managing dependency between objects (with dependency injection (DI)) Wiring the objects together Managing complete lifecycle of objects.

Q. Benefits of IOC.

Ans: Inversion of Control (IOC) principle is the base of Spring framework. It supports dependency injection in an application. With Dependency Injection, a programmer has to write minimal code. It also makes easier to test an application. Most important benefit is that it leads to loose coupling within objects. With loose coupling it is easier to change the application with new requirements.

IOC Container in Spring supports both the approaches. Eager instantiation as well as lazy loading of beans.

Q. What is Application Context?

Ans: A bean factory is fine to simple applications, but to take advantage of the full power of the Spring framework, you may want to move up to Spring's more advanced container, the application context. On the surface, an application context is same as a bean factory. Both load bean definitions, wire beans together, and dispense beans upon request. But it also provides:

- A means for resolving text messages, including support for internationalization.

- A generic way to load file resources.

- Events to beans that are registered as listeners.

Q. Implement ApplicationContext in Spring framework?

Ans: The 3 commonly used implementation of 'ApplicationContext' are

ClassPathXmlApplicationContext : It loads context definition from an XML file located in the classpath, treating context definitions as classpath resources. The application context is loaded from the application's classpath by using the code .

```
ApplicationContext context = new ClassPathXmlApplicationContext("bean.xml");
```

FileSystemXmlApplicationContext : It loads context definition from an XML file in the filesystem. The application context is loaded from the file system by using the code .

```
ApplicationContext context = new FileSystemXmlApplicationContext("bean.xml");
```

XmlWebApplicationContext : It loads context definition from an XML file contained within a web application.

Q. Difference between ApplicationContext and BeanFactory in Spring?

On the surface, an application context is same as a bean factory. But application context offers much more..

Application contexts provide a means for resolving text messages, including support for i18n of those messages.

Application contexts provide a generic way to load file resources, such as images.

Application contexts can publish events to beans that are registered as listeners.

Certain operations on the container or beans in the container, which have to be handled in a programmatic fashion with a bean factory, can be handled declaratively in an application context.

ResourceLoader support: Spring's Resource interface is a flexible generic abstraction for handling low-level resources. An application context itself is a ResourceLoader, Hence provides an application with access to deployment-specific Resource instances.

MessageSource support: The application context implements MessageSource, an interface used to obtain localized messages, with the actual implementation being pluggable

****Spring documentation recommends using ApplicationContext in almost all the cases. ApplicationContext has all the functionality of BeanFactory.**

Q. What is delegatingVariableResolver?

Ans: Spring provides a custom JavaServer Faces VariableResolver implementation that extends the standard Java Server Faces managed beans mechanism which lets you use JSF and Spring together. This variable resolver is called as DelegatingVariableResolver

Q. How to integrate Java Server Faces (JSF) with Spring?

Ans: JSF and Spring do share some of the same features, most noticeably in the area of IOC services. By declaring JSF managed-beans in the faces-config.xml configuration file, you allow the FacesServlet to instantiate that bean at startup. Your JSF pages have access to these beans and all of their properties. We can integrate JSF and Spring in two ways:

DelegatingVariableResolver: Spring comes with a JSF variable resolver that lets you use JSF and Spring together.

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
    "http://www.springframework.org/dtd/spring-beans.dtd">

<faces-config>
    <application>
        <variable-resolver>
            org.springframework.web.jsf.DelegatingVariableResolver
        </variable-resolver>
    </application>
</faces-config>
```

Q. What is Significance of JSF- Spring integration ?

Ans:It will be useful when an event handler wishes to explicitly invoke the bean factory to create beans on demand, such as a bean that encapsulates the business logic to be performed when a submit button is pressed.

Q. Main components of a typical Spring based application?

Ans:They are :

Spring configuration XML file: This is used to configure Spring application API

Interfaces: Definition of API interfaces for functions provided by application

Implementation: Application code with implementation of APIs Aspects: Spring Aspects implemented by application

Client: Application at client side that is used for accessing functions.

Q. Dependency Injection (DI)

Ans:Dependency Injection is a software design pattern. It is used to implement Inversion of Control (IOC) in Spring framework.

As per this pattern, we do not create objects in an application by calling new. Rather, we describe how an object should be created. In this way creation of an object is not tightly coupled with another object.

A container is responsible for creating and wiring the objects. The container can call injecting code and wire the objects as per the configuration at runtime.

Q.What are Roles in DI?

Ans:There are four roles in Dependency Injection:

Service object(s) to be used

Client object that depends on the service

Interface that defines how client uses services

Injector responsible for constructing services and injecting them into client

Q. Spring framework provides what kinds of Dependency Injection mechanism?

Ans:Constructor-based :Dependency Injection: Spring container can invoke a class constructor with a number of arguments. This represents a dependency on other class.

Setter-based Dependency Injection: Spring container can call setter method on a bean after creating it with a no-argument constructor or no-argument static factory method to instantiate another bean.

It is recommended to use Constructor-based DI for mandatory dependencies. Whereas Setter-based DI is used for optional dependencies.

Q.Advantages and Disadvantages of DI.

Ans: Dependency Injection (DI) pattern has following advantages: Dependency Injection reduces coupling between a class and its dependencies.

With Dependency Injection (DI), we can do concurrent or independent software development. Two teams can work parallel on classes that will be used by each other.

In DI, the client can be configured in multiple ways. It needs to just work with the given interface. Rest of the implementation can be changed and configured for different features.

It is also used to export a system's configuration details into configuration files. So we can configure same application run in different environments based on configuration. E.g. Run in Test environment, UAT environment, and Production environment.

It provide more ease and flexibility of testing. These can be tested in isolation in Unit Test. Dependency injection (DI) isolates client from the impact of design and implementation changes. Therefore, it promotes reusability, testability and maintainability.

Most of the time Dependency Injection forces developers to use an injection framework like Spring. This causes dependency on a framework. With Dependency Injection, clients are dependent on the configuration data. This becomes extra task for developers when the application does not need so many custom configuration values.

Code is difficult to trace and read in Dependency Injection. DI separates behavior from construction of objects. Dependency injection increases complexity in the linkages between classes. It may become harder to manage such complexity outside the implementation of a class.

Q. What is a Spring Bean?

Ans: A Spring Bean is a plain old Java object (POJO) that is created and managed by a Spring container. There can be more than one bean in a Spring application.

But all these Beans are instantiated and assembled by Spring container. Developer provides configuration metadata to Spring container for creating and managing the lifecycle of Spring Bean.

In general a Spring Bean is singleton. Every bean has an attribute named "singleton". If its value is true then bean is a singleton. If its value is false then bean is a prototype bean. By default the value of this attribute is true. Therefore, by default all the beans in spring framework are singleton in nature.

Q. What does the definition of a Spring Bean contain?

Ans: A Spring Bean definition contains configuration metadata for bean. This configuration metadata is used by Spring container to: Create the bean Manage its lifecycle Resolve its dependencies

Q. What are the different ways to provide configuration metadata to a Spring Container?

Ans: Spring supports three ways to provide configuration metadata to Spring Container: XML based configuration: We can specify configuration data in an XML file.

Annotation-based configuration: We can use Annotations to specify configuration. This was introduced in Spring 2.5. Java-based configuration: This is introduced from Spring 3.0. We can embed annotations like @Bean, @Import, @Configuration in Java code to specify configuration metadata.

Q. Different scopes of a Bean supported by Spring?

Ans: Spring framework supports seven types of scopes for a Bean. Out of these only five scopes are available for a web-aware ApplicationContext application: singleton: This is the default scope of a bean.

Under this scope, there is a single object instance of bean per Spring IoC container. prototype: Under this scope a single bean definition can have multiple object instances. request: In this scope, a single bean definition remains tied to the lifecycle of a single HTTP request. Each HTTP request will have its own instance of a bean for a single bean definition. It is only valid in the context of a web-aware Spring ApplicationContext.

session: Under this scope, a single bean definition is tied to the lifecycle of an HTTP Session. Each HTTP Session will have one instance of bean. It is also valid in the context of a web-aware Spring ApplicationContext.

globalSession: This scope, ties a single bean definition to the lifecycle of a global HTTP Session. It is generally valid in a Portlet context. It is also valid in the context of a web-aware Spring ApplicationContext.

application: This scope, limits a single bean definition to the lifecycle of a ServletContext. It is also valid in the context of a web-aware Spring ApplicationContext.

websocket: In this scope, a single bean definition is tied to the lifecycle of a WebSocket. It is also valid in the context of a web-aware Spring ApplicationContext.

Q. How to define the scope of a bean in Spring?

We can specify the scope of bean in its definition in configuration xml, . This is used by container to decide the scope of bean in Spring.

E.g. `< bean id = " userService" class = " com.atlanta.UserService" scope = " prototype" / >` This is an example of userService bean with prototype scope.

Q. Is a Singleton bean thread safe in spring?

Ans: No, Spring framework does not guarantee anything related to multi-threaded behavior of a singleton bean. Developer is responsible for dealing with concurrency issues and maintaining thread safety of a singleton bean.

Q. What are design Patterns used in spring.

Ans: Spring framework uses many Design patterns. Some of these patterns are: Singleton – By default beans defined in spring config files are singleton. These are based on Singleton pattern. Template – This pattern is used in many classes like- JdbcTemplate, RestTemplate, JmsTemplate, JpaTemplate etc.

Dependency Injection – This pattern is the core behind the design of BeanFactory and ApplicationContext.

Proxy – Aspect Oriented Programming (AOP) heavily uses proxy design pattern.

Front Controller – DispatcherServlet in Spring is based on Front Controller pattern to ensure that incoming requests are dispatched to other controllers.

Factory pattern – To create an instance of an object, BeanFactory is used. This is based on Factory pattern. View Helper – Spring has multiple options to separating core code from presentation in views. Like- Custom JSP tags, Velocity macros etc.

Q. What is the lifecycle of a Bean in Spring framework?

Ans: The different states in the lifecycle are:

Populate properties- Spring IoC container injects the bean's properties.

Set Bean Name- Spring container sets the bean name. If the bean implements `BeanNameAware`, spring container passes the bean's id to `setBeanName()` method.

Set Bean Factory-If the bean implements `BeanFactoryAware`, Spring container passes the `BeanFactory` to `setBeanFactory()`.

Pre Initialization-This stage is also called the bean postprocess . If there are any `BeanPostProcessors`, the Spring container calls the `postProcessorBeforeInitialization ()` method.

Initialize beans- If the bean implements `InitializingBean`, its `afterPropertySet()` method is called. If the bean has init method declaration, the specified initialization method is called.

Post Initialization- If `BeanPostProcessors` is implemented by the bean, the Spring container calls their `postProcessAfterInitialization()` method.

Ready to Use- Now the bean is ready to be used by the application.

Destroy- The bean is destroyed during this stage. If the bean implements `DisposableBean`, the Spring IoC container will call the `destroy()` method . If a custom `destroy ()` method is defined, the container calls the specified method.

Q. What are the two main groups of methods in a Bean's lifecycle?

Ans:It has two main groups of lifecycle methods.

Initialization Callbacks: Once all the necessary properties of a Bean are set by the container, Initialization Callback methods are used for performing initialization work. A developer can implement method `afterPropertiesSet()` for this work.

Destruction Callbacks: When the Container of a Bean is destroyed, it calls the methods in `DisposableBean` to do any cleanup work. There is a method called `destroy()` that can be used for this purpose to make Destruction Callbacks. Recent recommendation from Spring is to not use these methods, since it can strongly couple your code to Spring code.

Q. Can we override main lifecycle methods of a Bean in Spring?

Ans:Yes, Spring framework allows developers to override the lifecycle methods of a Bean. This is used for writing any custom behavior for Bean.

Q. Inner beans in Spring?

Ans:A bean that is used as a property of another bean is known as Inner bean.

It can be defined as a

< bean/ > element in < property/ > or < constructor-arg/ > tags.

It is not mandatory for an Inner bean to have id or a name. These are always anonymous. Inner bean does not need a scope. By default it is of prototype scope.

Q. How can we inject a Java Collection in Spring?

We used to pass single value at a time before the introduction of Collection. If we are required to pass multiple or bundle of data to be passed like list, set, and map etc. There are separate tags are used for these type of collection types as listed:

<list> : This tag is used to inject list of object.

<map> : This tag is used to inject bundled object as map.

<set> : This tag is used to inject object stored in Set.

<props> : This tag is used to inject collection of object in value - value pair fashion.

Q. Autowiring in Spring?

Ans: Autowiring is a feature of Spring in which container can automatically wire/ connect the beans by reading the configuration file. Developer has to just define "autowire" attribute in a bean. Spring resolves the dependencies automatically by looking at this attribute of beans that are autowired.

different modes of Autowiring supported by Spring:

There are five modes of Autowiring supported by Spring framework:

no: It is the default autowiring mode. It means no autowiring by default.

byName: The byName mode injects the object dependency according to name of the bean. In such case, property name and bean name must be same. It internally calls setter method.

byType: The byType mode injects the object dependency according to type. So property name and bean name can be different. It internally calls setter method.

constructor: The constructor mode injects the dependency by calling the constructor of the class. It calls the constructor having large number of parameters.

Q. Bean wiring in Spring?

Ans: A Spring container is responsible for injecting dependencies between beans. This process of connecting beans is called wiring. Developer mentions in configuration file, the dependencies between beans. And Spring container reads these dependencies and wires the beans on creation.

Q. Mention the cases in which Autowiring may fail in Spring framework?

Ans: Autowiring is a great feature in Spring. It can be used in most of the cases. But there are certain scenarios in which Autowiring may not work.

Explicit wiring: Since Autowiring is done by Spring, developer does not have full control on specifying the exact class to be used. It is preferable to use Explicit wiring in case of full control over wiring.
Primitive Data types: Autowiring does not allow wiring of properties that are based on primitive data types like- int, float etc.

Q. Is it allowed to inject null or empty String values in Spring?

Ans: Yes, Spring allows injecting null or empty String values.

Q. What is java based configuration in spring?

Ans: Spring allows for Java-based configuration in which a developer can specify configuration by using Java-based annotations.

We can use annotations like- @Configuration, @Bean, @Import and @DependsOn in Java classes for specifying the configuration.

Q. What is the purpose of @Configuration annotation?

Ans: This annotation is used in a class to indicate that this class is the primary source of bean definitions. This class can also contain inter-bean dependencies that are annotated by @Bean annotation.

Q. Difference between Full @Configuration and 'lite' @Beans mode?

Ans: Spring allows for using @Bean annotation on methods that are declared in classes not annotated with @Configuration. This is known as "lite" mode. In this mode, bean methods can be declared in a @Component or a plain java class without any annotation.

In the "lite" mode, @Bean methods cannot declare inter-bean dependencies. It is recommended that one @Bean method should not invoke another @Bean method in 'lite' mode.

Spring recommends that @Bean methods declared within @Configuration classes should be used for full configuration. This kind of full mode can prevent many bugs.

Q. In Spring framework, what is Annotation-based container configuration?

Ans: An alternative to XML setups is provided by annotation-based configuration which relies on the bytecode metadata for wiring up components instead of angle-bracket declarations. Instead of using XML to describe a bean wiring, the developer moves the configuration into the component class itself by using annotations on the relevant class, method, or field declaration.

Q. How will you switch on Annotation based wiring in Spring?

Ans: To use Annotation based wiring, we need to turn on Annotation based configuration in Spring. By default, Annotation based configuration is switched off in Spring. To turn it on we can specify <context:annotation-config/> element in Spring config file. Once it is turned on, we can use @Autowired annotation or @Required annotation in a Java class for wiring in Spring.

Q. What is @Autowired annotation?

Ans: We can use Spring @Autowired annotation for spring bean autowiring. @Autowired annotation can be applied on variables and methods for autowiring byType.

We can also use @Autowired annotation on constructor for constructor based spring autowiring. For @Autowired annotation to work, we also need to enable annotation based configuration in spring bean configuration file.

This can be done by context:annotation-config element or by defining a bean of type: org.springframework.beans.factory.annotation.AutowiredAnnotationBeanPostProcessor

Q. What is @Required annotation?

Ans: Sometimes, we need to make sure that all the required dependencies are set to the spring bean.

Spring provides @Required annotation to check spring dependencies. If the required dependencies are not set, then it throws BeanInitializationException.

Make sure to register `RequiredAnnotationBeanPostProcessor` in Spring config file when we use `@Required` annotation.

Q. What is `@Qualifier` annotation in Spring?

Ans:

`@Qualifier` annotation is used to mark a bean as ready for auto wiring.

This annotation is used along with `@Autowired` annotation to specify the exact bean for auto wiring by Spring container.

Q. How Spring framework makes JDBC coding part easier for developers?

Ans: Spring provides a mature JDBC framework to provide support for JDBC coding. Spring JDBC handled resource management as well as error handling in a generic way which reduces the work of software developers.

They just have to write queries and related statements to fetch the data or to store the data in database.

Q. What are the benefits of using Spring DAO?

Ans: Benefits of using Spring DAO are:

It makes it easier to work on different data access methods like- JDBC, Hibernate etc. It provides a consistent and common way to deal with different data access methods. Spring DAO makes it easier to switch between different data persistence frameworks. No need for catching framework specific exceptions.

Define connection parameters

Open the connection

Specify the statement

Prepare and execute the statement

Set up the loop to iterate through the results (if any)

Do the work for each iteration

Process any exception

Handle transactions

Close the connection

The Spring Framework takes care of all the grungy, low-level details that can make JDBC such a tedious API to develop with.

Q. What are the different ways to use Hibernate in Spring?

Ans: Spring provides two ways to use Hibernate: We can extend `HibernateDaoSupport` and apply an AOP interceptor node to use Hibernate. We can also use `HibernateTemplate` and `Callback` to access Hibernate. This is based on Inversion of Control.

Q. What types of Object Relational Mapping (ORM) are supported by Spring?

Ans: The Spring Framework supports integration with Hibernate, Java Persistence API (JPA) and Java Data Objects (JDO) for resource management, data access object (DAO) implementations, and transaction strategies.

For example, for Hibernate there is first-class support with several convenient IoC features that address many typical Hibernate integration issues.

You can configure all of the supported features for O/R (object relational) mapping tools through Dependency Injection. They can participate in Spring's resource and transaction management, and they comply with Spring's generic transaction and DAO exception hierarchies.

The recommended integration style is to code DAOs against plain Hibernate, JPA, and JDO APIs.

Q. How will you integrate Spring and Hibernate by using `HibernateDaoSupport`?

Ans: We can use following steps for integrating Spring and Hibernate: Add dependencies for Spring and Hibernate in `pom.xml`. Implement DAO from `HibernateDaoSupport`. Use Hibernate functions via `getHibernateTemplate()` method.

Q. What are the different types of the Transaction Management supported by Spring framework?

Ans: Spring framework provides support for 2 types of Transaction Management:

Programmatic: In this method, we have to manage Transaction by programming explicitly. It provides flexibility to a developer, but little bit complicated to maintain.

Declarative: In this approach, we can separate Transaction Management from the Application Business code. We can use annotations or XML based configuration to manage the transactions in declarative approach.

Q. What are the benefits provided by Spring Framework's Transaction Management?

Ans: Benefits provided by Spring Transaction Management are: Consistent programming model across different transaction APIs such as Java Transaction API (JTA), JDBC, Hibernate, Java Persistence API (JPA), and Java Data Objects (JDO).

Support for declarative transaction management.

Simpler API for programmatic transaction management than complex transaction APIs such as JTA.

Excellent integration with Spring's data access abstractions.

Q. Given a choice between declarative and programmatic Transaction Management, which method will you choose?

Ans: In Spring, Declarative Transaction Management is the preferred choice. This method is very less invasive and it has very less impact in Application Business Logic. Although Declarative method gives less flexibility than Programmatic method, it is simpler to use and easier to maintain in long run.

Q. What is Aspect Oriented Programming (AOP)?

Ans: It is an approach to programming that allows global properties of a program to determine how it is compiled into an executable program..

E.g. To develop payment software, one team can work on business logic for Money withdrawal, Money deposit, Money Transfer etc. The other team can work on Transaction Management for committing the transaction across multiple accounts.

Q. What is an Aspect in Spring?

Ans: An Aspect is the core construct of AOP. It encapsulates the behavior that affects multiple classes in a reusable module.

An Aspect can have a group of APIs that provide cross-cutting features. E.g. A logging module can be an Aspect in an Application.

An application can have multiple of Aspects based on the different requirements.

It can be implemented by using annotation `@Aspect` on a class.

Q. In Spring AOP, what is the main difference between a Concern and a Cross cutting concern?

Ans:

A Concern in Spring is the behavior or expectation from an application. It can be the main feature that we want to implement in the application.

A Cross cutting concern is also a type of Concern. It is the feature or functionality that is spread throughout the application in a thin way. E.g. Security, Logging, Transaction Management etc. are cross cutting concerns in an application.

Q. What is a Joinpoint in Spring AOP?

Joinpoint can be a method or an exception or a field getting modified. This is the place where the code of an Aspect is inserted to add new behavior in the existing execution flow.

Q. What is an Advice in Spring AOP?

Ans: This is an object which includes API invocations to the system wide concerns representing the action to perform at a joinpoint specified by a point.

Q. What are the different types of Advice in Spring AOP?

Ans: Spring AOP provides five kinds of Advice:

Before Advice: This type of advice runs just before a method executes. We can use `@Before` annotation for this.

After (finally) Advice: This type of advice runs just after a method executes. Even if the method fails, this advice will run.

We can use `@After` annotation here.

After Returning Advice: This type of advice runs after a method executes successfully.
@AfterReturning annotation can be used here.

After Throwing Advice: This type of advice runs after a method executes and throws an exception.
The annotation to be used is @AfterThrowing.

Around Advice: This type of advice runs before and after the method is invoked. We use @Around annotation for this.

Q. What is an Introduction in Spring AOP?

Ans: In Spring AOP we can declare additional methods or fields on behalf of a type.

To do this we use an Introduction. It is also known as inter-type declaration. E.g. We can use an Introduction for making a bean implement IsModified interface.

Q. What is a Target object in Spring AOP?

Ans: It is defined as the object that gets Advice from one or more Aspects. It is also known as advised object. In most cases it is a proxy object.

Q. What is a Pointcut in Spring AOP?

Ans: A pointcut defines at what joinpoints, the associated Advice should be applied. Advice can be applied at any joinpoint supported by the AOP framework

We can use class names, method names or regular expressions to specify the Pointcuts for an Advice.

Q. What is a Proxy in Spring AOP?

Ans: Spring AOP uses either JDK dynamic proxies or CGLIB to create the proxies for your target objects.

According to Spring documentation, in case your target implements at least one interface, a JDK dynamic proxy will be used. However if your target object does not implement any interfaces then a CGLIB proxy will be created.

Q. What are the different types of AutoProxy creators in Spring?

Ans: Spring AOP provides following standard types of Autoproxy creators:

BeanNameAutoProxyCreator: This is a BeanPostProcessor that creates AOP proxies for beans automatically by matching names.

DefaultAdvisorAutoProxyCreator: This creator is more powerful than other Proxy Creators. This also applies eligible advisors automatically to bean in the current context.

AbstractAdvisorAutoProxyCreator: This is the parent class of DefaultAdvisorAutoProxyCreator. We can create our own auto-proxy creators by extending this class.

Q. What is Weaving in Spring AOP?

Ans: In Spring AOP makes it possible to modularize and separate logging, transaction like services and apply them declaratively to the components. Hence programmer can focus on specific concerns. Aspects are wired into objects in the spring XML file in the way as JavaBean. This process is known as 'Weaving'

Q. In Spring AOP, Weaving is done at compile time or run time?

Ans:Run time.

Q. What is XML Schema-based Aspect implementation?

Ans:Spring allows for implementing Aspect by using regular classes and XML based configurations. This is different from Annotation based Aspect implementation. But it achieves the same goal of AOP.

We can use elements like

< aop:aspect id =" testAspect" ref =" testBean" /> and < aop:pointcut id =" testPointcut" /> in Spring XML config file.

To use this we need to import Spring AOP schema as follows:

< beans xmlns =" http:// www.springframework.org/ schema/ beans" xmlns:xsi =" http:// www.w3.org/ 2001/ XMLSchema-instance"

xmlns:aop =" http:// www.springframework.org/ schema/ aop" xsi:schemaLocation =" http:// www.springframework.org/ schema/ beans http:// www.springframework.org/ schema/ beans/ spring-beans-3.0. xsd http:// www.springframework.org/ schema/ aop http:// www.springframework.org/ schema/ aop/ spring-aop-3.0.xsd">

Q. What is Annotation-based aspect implementation in Spring AOP?

Ans:This is a declarative style AOP implementation. In this case, we use annotations like @Aspect, @Pointcut, @Joinpoint etc. to annotate code with different types of AOP elements. This can be used Java 5 onwards, when the support for Annotations was introduced.

Q. How does Spring MVC framework work?

Ans:Spring provides its own Model View Controller (MVC) framework for developing web applications. Spring MVC framework is based on Inversion of Control (IOC) principle. It separates the business objects from controller. It is designed around the DispatcherServlet that is responsible for dispatching requests to relevant handlers. Spring MVC framework also supports annotation based binding of request parameters.

Q. What is DispatcherServlet?

Ans:In Spring MVC, DispatcherServlet is the core servlet that is responsible for handling all the requests and dispatching these to handlers.

Dispatcher servlet knows the mapping between the method to be called and the browser request.

It calls the specific method and combines the results with the matching JSP to create an html document, and then sends it back to browser.

In case of RMI invocation, it sends back response to the client application.

Q. Can we have more than one DispatcherServlet in Spring MVC?

Ans:Yes, a Spring MVC web application can have more than one DispatcherServlets.

Each DispatcherServlet has to operate in its own namespace. It has to load its own ApplicationContext with mappings, handlers, etc. Only the root application context will be shared among these Servlets.

Q.What is WebApplicationContext in Spring MVC?

Ans:WebApplicationContext is the child of plain ApplicationContext. It is used in web applications.

It provides features to deal with web-related components like- controllers, view resolvers etc.

A Web Application can have multiple WebApplicationContext to handle requests.

Each DispatcherServlet is associated with one WebApplicationContext.

Q. What is Controller in Spring MVC framework?

Ans:It is an interface in Spring MVC. It receives HttpServletRequest and HttpServletResponse in web app just like an HttpServlet, but it is able to participate in an MVC flow.

They are similar to a Struts Action in a Struts based Web application. Spring recommends that the implementation of Controller interface should be a reusable, thread-safe class, capable of handling multiple HTTP requests throughout the lifecycle of an application.

It is preferable to implement Controller by using a JavaBean. Controller interprets user input and transforms it into a model. The model is represented to the user by a view. Spring implements a controller in a very generic way. This enables us to create a wide variety of controllers.

We use @ Controller annotation to indicate that a class is a Controller in Spring MVC. The dispatcher in Spring scans for @Controller annotated classes for mapped methods and detects @RequestMapping.

Q. What is @RequestMapping annotation in Spring?

Ans:In Spring MVC, we use @RequestMapping annotation to map a web request to either a class or a handler method.

Annotation for mapping web requests onto specific handler classes and/or handler methods. Provides a consistent style between Servlet and Portlet environments, with the semantics adapting to the concrete environment.

In @RequestMapping we can specify the path of URL as well as HTTP method like- GET, PUT, POST etc. @RequestMapping also supports specifying HTTP Headers as attributes.

We can also map different media types produced by a controller in @RequestMapping. We use HTTP Header Accepts for this purpose. E.g. @RequestMapping(value = "/ test/ mapping", method = GET, headers = "Accept = application/ json")

Q. What are the main features of Spring MVC?

Ans: The Spring Web MVC framework provides Model-View-Controller (MVC) architecture and ready components that can be used to develop flexible and loosely coupled web applications. The MVC pattern results in separating the different aspects of the application (input logic, business logic, and UI logic), while providing a loose coupling between these elements.

The Model encapsulates the application data and in general they will consist of POJO.

The View is responsible for rendering the model data and in general it generates HTML output that the client's browser can interpret.

The Controller is responsible for processing user requests and building an appropriate model and passes it to the view for rendering.

Spring MVC has following main features: Clear separation of role: In Spring MVC, each role like-controller, validator, command object, form object, model object, DispatcherServlet, handler mapping, view resolver etc. is fulfilled by a specialized object.

Reusability: Spring MVC promotes reusable business code that reduces the need for duplication. We can use existing business objects as command or form objects instead of copying them to extend a particular framework base class.

Flexible Model Transfer: Spring MVC Model transfer supports easy integration with other view technologies as well.

Customizable binding and validation: In Spring MVC, we can do custom binding between Requests and Controllers. Even validation can be done on non-String values as well

Customizable locale, time zone and theme resolution: Spring MVC supports customization in locale, timezone etc.

Q. What is the difference between a Singleton and Prototype bean in Spring?

Ans: Scope defines the existence timeframe of beans in the application. Singleton scope for bean limits a bean to a single object instance per Spring IOC container. This single instance is limited to a specific ApplicationContext.

If there are multiple ApplicationContext then we can have more than one instance of bean. By default all the beans in Spring framework are Singleton scope beans.

With Prototype scope a single bean definition can have multiple object instances in a Spring container. In prototype scope bean, the Spring IoC container creates new bean instance of the object every time a request for that specific bean is made.

Q. How will you decide which scope-Prototype or Singleton to use for a bean in Spring?

Ans: We use prototype scope for all stateful beans and singleton scope for stateless beans.

Since a stateless bean does not maintain any state, we can use the same object instance again and again. Singleton scope bean serves the same purpose.

In a stateful bean, there is a need to maintain the state in each request, it is necessary to use a new instance of object with each call.

A Prototype scope bean ensures that we get a new instance each time we request for the object.

Q. difference between Setter and Constructor based Dependency Injection (DI) in Spring framework?

Ans:

Priority: Setter based injection has higher priority than a constructor based injection in Spring.

If an application uses Setter as well as Constructor injection, Spring container uses the Setter injection.

Partial dependency: We can inject partial dependency by using Setter injection. In Constructor injection, it is not possible to do just a partial dependency injection. E.g. If there are two properties in a class, we can use Setter method to inject just one property in the class.

Flexibility: Setter injection gives more flexibility in introducing changes. One can easily change the value by Setter injection. In case of Constructor injection a new bean instance has to be created always.

Readability: Setter injection is more readable than Constructor injection. Generally Setter method name is similar to dependency class being used in setter method.

Q. What are the drawbacks of Setter based Dependency Injection (DI) in Spring?

Ans: Although Setter based Dependency Injection has higher priority than Constructor based DI, there are some disadvantages of it.

No Guarantee: In Setter based DI, there is no guarantee that a certain dependency is injected or not. We may have an object with partial or no dependency. Whereas in Constructor based DI, an object is not created till the time all the dependencies are ready.

Security: One can use Setter based DI to override another dependency. This can cause Security breach in a Spring application.

Circular Dependency: Setter based DI can cause circular dependency between objects. Whereas Constructor based DI will throw `ObjectCurrentlyInCreationException` if there is a circular dependency during the creation of an object.

Q. What is the difference between `FileSystemResource` and `ClassPathResource`?

Ans: In Spring we can specify configuration by using a file or classpath.

In `FileSystemResource` we have to give absolute path / relative path of Spring Configuration file `spring-config.xml` file.

In `ClassPathResource` Spring looks for Spring Configuration file `spring-config.xml` in ClassPath. Therefore, developer has to include `spring-config.xml` in classpath.

`ClassPathResource` searches for configuration file in CLASSPATH, whereas `FileSystemResource` searches for configuration file in file system.

Q. Name some popular Spring framework annotations that you use in your project?

Ans: Few are:

@Controller: For creating controller classes in a Spring MVC project.

@RequestParam: Binds a request parameter to a method parameter.

@SessionAttributes: Specifies that a model attribute should be stored in the session.

@RequestMapping: Maps a URL pattern and/or HTTP method to a method or controller type.

@ResponseBody: For sending an Object as response we use this annotation.

@PathVariable: To map dynamic values from a URI to handler method arguments, we use this annotation.

@Autowired: Declares a constructor, field, setter method, or configuration method to be autowired by type. Items annotated with @Autowired do not have to be public.

@Scope: Specifies the scope of a bean, either singleton, prototype, request session, or some custom scope..

@Configuration: This is an annotation for Java based Spring configuration.

@Aspect, @Before, @After, @Around, @Joinpoint, @Pointcut: These are the annotations in Spring for AspectJ AOP

Q. How can you upload a file in Spring MVC Application?

Ans: We can use MultipartResolver interface to upload a file. We need to make configuration changes to make it work.

After uploading the file, we have to create Controller handler method to process the uploaded file in application.

Q. Different types of events provided by Spring framework?

Ans: Spring framework provides following five events for Context:

ContextRefreshedEvent: Whenever ApplicationContext is initialized or refreshed, Spring publishes this event. We can also raise it by using refresh() method on ConfigurableApplicationContext interface.

ContextStartedEvent: When ApplicationContext is started using start() method on ConfigurableApplicationContext interface, ContextStartedEvent is published. We can poll database or restart any stopped application after receiving this event.

ContextStoppedEvent: Spring publishes this event when ApplicationContext is stopped using stop() method on ConfigurableApplicationContext interface. This is used for doing any cleanup work.

ContextClosedEvent: Once the ApplicationContext is closed using close() method, ContextClosedEvent is published. Once a context is closed, it is the last stage of its lifecycle. After this it cannot be refreshed or restarted.

RequestHandledEvent: This is a web specific event that informs to all beans that an HTTP request has been serviced.

Q. What is the difference between DispatcherServlet and ContextLoaderListener in Spring?

Ans:DispatcherServlet is the core of Spring MVC application. It loads Spring bean configuration file and initialize all the beans mentioned in config file.

In case we have enabled annotations in Spring config file, it also scans the packages and configures any bean annotated with @Component, @Controller, @Repository or @Service annotations.

ContextLoaderListener is a listener to start up and shut down Spring's root WebApplicationContext. It links the lifecycle of ApplicationContext to the lifecycle of the ServletContext. It automates the creation of ApplicationContext. It can also be used to define shared beans used across different spring contexts.

Q. How will you handle exceptions in Spring MVC Framework?

Ans:Spring MVC Framework provides following mechanisms to help us achieve exception handling:

Controller Based: A developer can define exception handler methods in a Controller class. To do so, they have to annotate the methods with @ExceptionHandler annotation.

Global Exception Handler: Spring provides @ControllerAdvice annotation for exception handling as cross-cutting concern. We can mark any class as global exception handler by using this annotation.

HandlerExceptionResolver implementation: Spring Framework provides HandlerExceptionResolver interface that can be implemented to create a global exception handler.

Q. What is Spring Security ?

Ans: Spring security is a project under spring framework umbrella.

It provides support for security requirements of enterprise Java projects.

It provides out of box support for creating login screen, remember me cookie support, securing URL, authentication provider to authenticate the user from the database and in memory, concurrent active session management support and much more.

In order to use it in Spring MVC based project, you need to include spring-security.jar and configure it in application-Context-security.XML file, you can name it whatever you want, but make sure to supply this to ContextLoaderListener, which is responsible for creating Spring context and initializing dispatcher servlet.

Q. How to limit Number of User Session?

Ans:

Add the below code in applicaContext-security.xml

```
<session-management invalid-session-url="/logout.html">
```

```
<concurrency-control max-sessions="1"
```

```
error-if-maximum-exceeded="true" />  
</session-management>
```

Session-management element is used to capture session related stuff. Max-session specify how many concurrent authenticated session is allowed and if error-if-maximum-exceeded set to true it will flag error if user tries to login into another session.

Hibernate

Q. What is Hibernate framework?

Ans: Hibernate ORM enables developers to more easily write applications whose data outlives the application process. As an Object/Relational Mapping (ORM) framework, Hibernate is concerned with data persistence as it applies to relational databases (via JDBC).

In addition to its own "native" API, Hibernate is also an implementation of the Java Persistence API (JPA) specification. As such, it can be easily used in any environment supporting JPA including Java SE applications, Java EE application servers, Enterprise OSGi containers, etc.

Hibernate enables you to develop persistent classes following natural Object-oriented idioms including inheritance, polymorphism, association, composition, and the Java collections framework. Hibernate requires no interfaces or base classes for persistent classes and enables any class or data structure to be persistent.

High Performance: Hibernate supports lazy initialization, numerous fetching strategies and optimistic locking with automatic versioning and time stamping. Hibernate requires no special database tables or fields and generates much of the SQL at system initialization time instead of at runtime.

Scalability: Hibernate was designed to work in an application server cluster and deliver a highly scalable architecture.

Extensibility: Hibernate is highly configurable and extensible

Reliable: Hibernate is well known for its excellent stability and quality, proven by the acceptance and use by tens of thousands of Java developers

Q. What is an Object Relational Mapping (ORM)?

Ans: Object Relational Mapping (ORM) is a programming technique to map data from a relational database to Object oriented domain model. This is the core of Hibernate framework. In case of Java, most of the software is based on OOPS design. But the data stored in Database is based on Relation Database Management System (RDBMS). ORM helps in data retrieval in an Object Oriented way from an RDBMS. It reduces the effort of developers in writing queries to access and insert data.

Q. what is persistence?

Ans: Persistence simply means that we would like our application's data to outlive the applications process. In Java terms, we would like the state of (some of) our objects to live beyond the scope of the JVM so that the same state is available later.

Q. Purpose of Configuration Interface in Hibernate?

Ans: Configuration interface can be implemented in an application to specify the properties and mapping documents for creating a SessionFactory in Hibernate. By default, a new instance of Configuration uses properties mentioned in hibernate.properties file.

Configuration is mainly an initialization time object that loads the properties in helps in creating SessionFactory with these properties. In short, Configuration interface is used for configuring Hibernate framework in an application.

Q.What is Object-Relational Impedance Mismatch?

Ans:

'Object-Relational Impedance Mismatch' (sometimes called the 'paradigm mismatch') is just a fancy way of saying that object models and relational models do not work very well together. RDBMSs represent data in a tabular format (a spreadsheet is a good visualization for those not familiar with RDBMSs), whereas object-oriented languages, such as Java, represent it as an interconnected graph of objects. Loading and storing graphs of objects using a tabular relational database exposes us to 5 mismatch problems.

Granularity

Sometimes you will have an object model which has more classes than the number of corresponding tables in the database (we say the object model is more granular than the relational model). Take for example the notion of an Address...

Subtypes (inheritance)

Inheritance is a natural paradigm in object-oriented programming languages. However, RDBMSs do not define anything similar on the whole (yes some databases do have subtype support but it is completely non-standardized)...

Identity

A RDBMS defines exactly one notion of 'sameness': the primary key. Java, however, defines both object identity $a == b$ and object equality $a.equals(b)$.

Associations

Associations are represented as unidirectional references in Object Oriented languages whereas RDBMSs use the notion of foreign keys. If you need bidirectional relationships in Java, you must define the association twice.

Likewise, you cannot determine the multiplicity of a relationship by looking at the object domain model.

Data navigation

The way you access data in Java is fundamentally different than the way you do it in a relational database. In Java, you navigate from one association to another walking the object network.

This is not an efficient way of retrieving data from a relational database. You typically want to minimize the number of SQL queries and thus load several entities via JOINS and select the targeted entities before you start walking the object network.

Q. Interfaces of Hibernate framework?

Ans: What are the core interfaces of Hibernate framework?

1.

Session Interface:

The basic interface for all hibernate applications. The instances are lightweighted and can be created and destroyed without expensive process.2.

SessionFactory interface:

The delivery of session objects to hibernate applications is done by this interface. For the whole application, there will be generally one SessionFactory and can be shared by all the application threads.3.

Configuration Interface:

• hibernate bootstrap action is configured by this interface. The location specification is specified by specific mapping documents, is done by the instance of this interface."

Transaction Interface:

This is an optional interface. This interface is used to abstract the code for a transaction that is implemented such as a `beginTransaction()`.

Query and Criteria interface:

The queries for the user are allowed by this interface apart from controlling the flow of the query execution

Q. What is the use of session in hibernate?

Ans: It is used to get a physical connection with a database. The Session object is lightweight and designed to be instantiated each time an interaction is needed with the database. Persistent objects are saved and retrieved through a Session object.

Q. How will you map the columns of a DB table to the properties of a Java class in Hibernate?

Ans: We can do this in 2 ways:

XML: We can map the column of a table to the property of a class in XML file. It is generally with extension hbm.xml

Annotation: We can also use annotations @Entity and @Table to map a column to the property of a class.

Q. What are the steps for creating a SessionFactory in Hibernate?

Ans: Configuration: First create a Configuration object. This will refer to the path of configuration file.

Resource: Add config file resource to Configuration object. Properties: Set properties in the Configuration object.

SessionFactory: Use Configuration object to build SessionFactory.

Eg. Configuration config = new Configuration(); config.addResource("testInstance/configuration.hbm.xml");

config.setProperties(System.getProperties());

SessionFactory sessions = config.buildSessionFactory();

Q. Why do we use POJO in Hibernate?

Ans: POJO stands for Plain Old Java Objects. A POJO is a Java bean with getter and setter methods for each property of the bean.

It is a simple class that encapsulates an object's properties and provides access through setters and getters.

Some of the reasons for using POJO in Hibernate are: POJO emphasizes the fact that this class is a simple Java class, not a heavy class like EJB.

POJO is a well-constructed class, so it works well with Hibernate proxies. POJO also comes with a default constructor that makes it easier to persist with a default constructor.

Q. Explain Object states in Hibernate

Ans: A persistent class object can be in one of the following three states:

1. Transient.
2. Persistent.
3. Detached

Q. Difference between openSession and getCurrentSession?

Ans:The `SessionFactory.getCurrentSession()` returns a session bound to a context. We don't need to close this. We have to set `hibernate.current_session_context_class` to `thread` and then implement something like a servlet filter that opens the session. We can access that session anywhere else by using the `SessionFactory.getCurrentSession().thread`

The `SessionFactory.openSession()` always opens a new session that we have to `flush()` and `close()` once we are done with the operations

Q.HQL(Hibernate Query Language)?

Ans:HQL stands for Hibernate Query Language. HQL syntax is quite similar to SQL syntax but it performs operations on objects and properties of persistent classes instead of tables and columns. Hibernate framework translate HQL queries into database specific queries to perform action.

Q. Can we make an hibernate entity class final?

Ans:Yes, we can make a Hibernate Entity class final, but that's not a good practice.

Since, hibernate use proxy classes for lazy loading of data which is done by extending the entity bean. If the entity bean will be final then lazy loading will not be possible which results into low performance.

Q. What will happen if we don't have no-args constructor in Entity bean?

Ans:

Hibernate uses Reflection API to create instance of Entity beans. The method `Class.newInstance()` is used for creating the instance of Entity beans which requires no-args constructor.

So if we won't have no-args constructor in entity beans, hibernate will fail to instantiate it and we will get `HibernateException`.

Q. What is the difference between `session.merge` vs `session.update` in hibernate?

Ans:Update and Merge both work on detached instances, ie instances which have a corresponding entry in the database but which are currently not attached to a Session. Both methods are used to convert these object into persistence state from detached state.

The update tries to reattach the instance, that means that there may be no other instance of the persistent entity attached to the Session right now otherwise an exception is thrown. i.e. update method cannot be used when the same object exists in the session.

The merge copies all values to a persistent instance in the session. The input object is not changed. So merge is more general than update, but may use more resources. i.e. The merge method will work fine when the same object exists in the session.

Q. What is the difference between `session.save` and `session.persist` method in hibernate?

Ans:Save():

1. Syntax: `public Serializable save(Object object)` throws `HibernateException`.
2. The return type of the save method is `java.io.Serializable`. It returns the generated Id after saving the record.
3. It can save the changes to the database outside of the transaction. i.e. Save method can be used inside or outside the transaction.
4. `Session.save()` for a detached object will create a new row in the table.
5. Save method takes more time to execute.

`Persist()`:

1. Syntax: `public void persist(Object object)` throws `HibernateException`.
2. It does not return generated Id after saving. Its return type is `void`.
3. Persist method can be used only within the transaction.
4. `session.persist()` for a detached object will throw `PersistentObjectException` as it is not allowed.
5. Persist method takes less time to execute.

Q. How will you call a stored procedure in Hibernate?

Ans: Hibernate supports executing not only simple queries but also stored procedure of database. There are three ways to call a stored procedure in Hibernate: XML mapping file: We can declare the store procedure inside XML Mapping file.

Native SQL: We can use Native SQL to call a store procedure query directly.

Use annotation: We can also mark out stored procedure with `@NamedNativeQueries` annotation.

Q. What is Criteria API in Hibernate?

Ans: Hibernate Criteria API is a more object oriented and elegant alternative to Hibernate Query Language (HQL). It's always a good solution to an application which has many optional search criteria.

Let go through some popular Criteria API functions.

Criteria basic query

eg: `Criteria criteria = session.createCriteria(StockDailyRecord.class);`

Q. How can you see SQL code generated by Hibernate on console?

Turn on the `show_sql` flag. This can be done in Hibernate configuration as follows: `< property name="show_sql" > true </ property >`

Q. What are the different types of collections supported by Hibernate?

Ans: Hibernate supports following two types of collections: Indexed Collections: List and Maps
Sorted Collections: `java.util.SortedMap` and `java.util.SortedSet`.

Q. What is the difference between `session.save()` and `session.saveOrUpdate()` methods in Hibernate?

Ans: Save:

Save

`Serializable save(Object object)` throws `HibernateException`

`Serializable save(String entityName,
Object object)`
throws `HibernateException`

Save method is used to store an object into the database. It insert an entry if the identifier doesn't exist. It will throw error if the identifier already exist.

Update

`void update(Object object)` throws `HibernateException`

`void update(String entityName, Object object)` throws `HibernateException`

Update method in the hibernate is used for updating the object using identifier. It will throw an exception if the identifier is missing or doesn't exist.

`saveOrUpdate`

`void saveOrUpdate(Object object)` throws `HibernateException` `void saveOrUpdate(String entityName, Object object)` throws `HibernateException`

The `saveOrUpdate` method calls `save()` or `update()` method based on the operation. If the identifier exists, it will call update method else the save method will be called.

Q. How can we get statistics of a SessionFactory in Hibernate?

Ans: The first thing we need to do is tell Hibernate to generate statistics for us. Therefore we need to set the system property `hibernate.generate_statistics` to true. Statistics are deactivated by default because it has a bad influence on the performance. So only activate them when you need them.

Now Hibernate will write one multi-line log statement with summarized statistical information at the end of the session. This is quite useful to get an overview on how much time was spend on database communication. But in most cases we need more information. We need to know, if there was a specific query that took a long time or if we have an overall performance issue. Therefore we would like to log the execution time for each query. This can be done by setting the log level for `org.hibernate.stat` to `DEBUG`.

Q. What is the Transient state of an object in Hibernate?

Ans: When an object is just instantiated using the new operator but is not associated with a Hibernate Session, then the object is in Transient state.

In Transient state, object does not have a persistent representation in database. Also there is no identifier assigned to an object in Transient state. An object in Transient state can be garbage collected if there is no reference pointing to it.

Q. What is the Detached state of an object in Hibernate?

Ans: An object is in detached state if it was persistent earlier but its Session is closed now. Any reference to this object is still valid. We can even update this object. Later on we can even attach an object in detached state to a new session and make it persistent. Detached state is very useful in application transactions where a user takes some time to finish the work.

Q. What is the use of Dirty Checking in Hibernate?

Ans: Hibernate monitors all persistent objects (will discuss in a while). At the end of a unit of work, it knows which objects have been modified. Then it calls update statement on all updated objects. This process of monitoring and updating only objects that have been changed is called automatic dirty checking in Hibernate.

Q. What is the purpose of Callback interface in Hibernate?

Ans: Callback interfaces allow the application to receive a notification when something interesting happens to an object—for example, when an object is loaded, saved, or deleted. Hibernate applications don't need to implement these callbacks, but they're useful for implementing certain kinds of generic functionality.

Q. Query Cache in Hibernate?

Ans: Hibernate provides Query Cache to improve the performance of queries that run multiple times with same parameters. At times Query Caching can reduce the performance of Transactional processing. By default Query Cache is disabled in Hibernate.

It has to be used based on the benefits gained by it in performance of the queries in an application.

Q. Unit of Work design pattern?

Ans: Unit of Work design pattern does two important things: first it maintains in-memory updates and second it sends these in-memory updates as one transaction to the database.

So to achieve the above goals it goes through two steps:

It maintains lists of business objects in-memory which have been changed (inserted, updated, or deleted) during a transaction.

Once the transaction is completed, all these updates are sent as one big unit of work to be persisted physically in a database in one go.

Q. How does Transaction management work in Hibernate?

Ans: The idea is that you group database operations into a single transaction and either they're all successful or they all fail. This way your database cannot end up in an intermediate and invalid state.

Transaction management is a vast and often quite complex area and the way in which you configure it depends on your specific application setup.

```
Session s = null; Transaction trans = null; try { s = sessionFactory.openSession(); trans = s.beginTransaction(); doTheAction( s); trans.commit(); } catch (RuntimeException exc) { trans.rollback(); } finally {s.close(); }
```

Q. How can we mark an entity/ collection as immutable in Hibernate?

Ans: In Hibernate, by default an entity or collection is mutable. We can add, delete or update an entity/ collection.

To mark an entity/ collection as immutable, we can use one of the following: @Immutable: We can use the annotation @Immutable to mark an entity/ collection immutable. XML file: We can also set the property mutable = false in the XML file for an entity to make it immutable.

Q. What are the different options to retrieve an object from database in Hibernate?

Ans: Hibernate provides three different ways to retrieve data from database. HQL and native SQL queries and Criteria. The Criteria Query API lets you build nested, structured query expressions in Java, providing a compile-time syntax checking that is not possible with a query language like HQL or SQL. The Criteria API also includes query by example (QBE) functionality. This lets you supply example objects that contain the properties you would like to retrieve instead of having to step-by-step spell out the components of the query. It also includes projection and aggregation methods, including count()

Q. How can we auto-generate primary key in Hibernate?

Ans: We can use the primary key generation strategy of type GenerationType.AUTO to auto-generate primary key while persisting an object in Hibernate.

Eg. @Id @GeneratedValue(strategy = GenerationType.AUTO) private int id; We can leave it null/ 0 while persisting and Hibernate automatically generates a primary key for us. Sometimes, AUTO strategy refers to a SEQUENCE instead of an IDENTITY

Q. What is the first level of cache in Hibernate?

Ans: Session is the first level of cache in Hibernate.

First-level cache:

The first-level cache is the Session cache and is a mandatory cache through which all requests must pass. The Session object keeps an object under its own power before committing it to the database.

If you issue multiple updates to an object, Hibernate tries to delay doing the update as long as possible to reduce the number of update SQL statements issued. If you close the session, all the objects being cached are lost and either persisted or updated in the database.

Second-level cache:

Second level cache is an optional cache and first-level cache will always be consulted before any attempt is made to locate an object in the second-level cache. The second-level cache can be configured on a per-class and per-collection basis and mainly responsible for caching objects across sessions.

Any third-party cache can be used with Hibernate. An `org.hibernate.cache.CacheProvider` interface is provided, which must be implemented to provide Hibernate with a handle to the cache implementation.

Query-level cache:

Hibernate also implements a cache for query resultsets that integrates closely with the second-level cache.

This is an optional feature and requires two additional physical cache regions that hold the cached query results and the timestamps when a table was last updated. This is only useful for queries that are run frequently with the same parameters.

The Second Level Cache:

Hibernate uses first-level cache by default and you have nothing to do to use first-level cache. Let's go straight to the optional second-level cache. Not all classes benefit from caching, so it's important to be able to disable the second-level cache

The Hibernate second-level cache is set up in two steps. First, you have to decide which concurrency strategy to use. After that, you configure cache expiration and physical cache attributes using the cache provider.

Q. Default transaction factory in Hibernate?

Ans: `JDBCTransactionFactory`. But we can change it by setting the property `hibernate.transaction.factory_class`.

Q. What are the options to disable second level cache in Hibernate?

Ans: By default it is disabled.

We can set `hibernate.cache.use_second_level_cache` to false. We can use `CacheMode.IGNORE` to stop interaction between the session and second-level cache. Session will interact with cache only to invalidate cache items when updates occur

Q. What is the difference between Immediate fetching and Lazy collection fetching?

Ans:

`FetchType.LAZY` = Doesn't load the relationships unless explicitly "asked for" via getter

`FetchType.EAGER` = Loads ALL relationships

Lazy fetching is better from performance perspective.

Q. What is 'Extra lazy fetching' in Hibernate?

Ans: In Extra lazy fetching, only individual elements of a collection are fetched from the database when they are required.

Q. How can we check if a collection is initialized or not under Lazy Initialization strategy?

Ans: Hibernate provides two convenient methods, `Hibernate.initialize()` and `Hibernate.isInitialized()` to check whether a collection is initialized or not. By using `Hibernate.initialize()` we can force the initialization of a collection in Hibernate.

Q. different strategies for cache mapping in Hibernate?

Ans:

Read only:

Read/ Write:

Read/ write cache strategy should not be used if there is requirement for serializable transaction isolation level.

Nonstrict Read/ Write: If an application only occasionally updates the data, then we can use this strategy. It cannot be used in systems with serializable transaction isolation level requirement.

Transactional: This strategy supports full transactional cache providers like JBoss TreeCache.

Q. difference between a List vs Set vs Bag in Hibernate?

Ans:

List: Ordered collection of entities, duplicate allowed. Use a `.NET IList` in code. The index column will need to be mapped in NHibernate.

Set: Unordered collection of unique entities, duplicates not allowed. Use `java.util.Collection.ISet` in code (NH prior to v4) or `System.Collections.Generic.ISet` (NH v4+). It is important to override `GetHashCode` and `Equals` to indicate the business definition of duplicate. Can be sorted by defining an `orderby` or by defining a comparer resulting in a `SortedSet` result.

Bag: Unordered list of entities, duplicates allowed. Use a `.NET ICollection<T>` in code. The index column of the list is not mapped and not honored by NHibernate.

Q. How can we check if an Object is in Persistent, Detached or Transient state in Hibernate?

Ans: Persistent State: If call to `EntityManager.contains( object)` returns true, the object is in Persistent state. Detached State: If the call to `PersistenceUnitUtil.getIdentifier( object)` returns identifier property then the object is in detached state. Transient State: If call to `PersistenceUnitUtil.getIdentifier( object)` returns null then object is in Transient state. We can get access to `PersistenceUnitUtil` from the `EntityManagerFactory` in Hibernate.

Q. What is 'the inverse side of association' in a mapping?

Ans: Let us consider an example in which a customer can have multiple orders and for every order there has to be a customer. In OO world, customer is the owner of order. In SQL world, an Order has reference to customer id. It is a bi-directional one to many mapping from customer to order. The inverse side in this mapping is the owner of object. In this case customer is the owner of order. Since an order cannot exist without a customer. But a customer can exist without an order. Also customer has no column to save order data. But an Order table can store customer id, which is used for mapping.

Q. What is the difference between `load()` and `get()` method in Hibernate?

Ans:.

1. session.load()

It will always return a "proxy without hitting the database. In Hibernate, proxy is an object with the given identifier value, its properties are not initialized yet, it just look like a temporary fake object.

If no row found , it will throws an ObjectNotFoundException.

2. session.get()

It always hit the database and return the real object, an object that represent the database row, not proxy.

If no row found , it return null.

Q. What is a derived property in Hibernate?

Ans:In Hibernate, a derived property is not mapped to any column of a database table. A derived property is computed at runtime by evaluation of an expression. These are read only properties.

Egg. In this example profitMargin is derived from salePrice and buyPrice. < property name ="profitMargin" formula ="(SELECT (i.salePrice – i.buyPrice) FROM item i WHERE i.Id = Id)"/ >

Q. How can we use Named Query in Hibernate?

Ans:The hibernate named query is way to use any query by some meaningful name. It is like using alias names. The Hibernate framework provides the concept of named queries so that application programmer need not to scatter queries to all the java code.

There are two ways to define the named query in hibernate:

a.by annotation

b.by mapping file.

Hibernate Named Query by annotation

If you want to use named query in hibernate, you need to have knowledge of @NamedQueries and @NamedQuery annotations.

@NameQueries annotation is used to define the multiple named queries.

@NamedQuery annotation is used to define the single named query.

Let's see the example of using the named queries:

```
@NamedQueries(  
    {  
        @NamedQuery(  
            name = "findEmployeeByName",  
            query = "from Employee e where e.name = :name"
```

```
)  
}  
)
```

Hibernate Named Query by mapping file

If want to define named query by mapping file, you need to use query element of hibernate-mapping to define the named query.

In such case, you need to create hbm file that defines the named query. Other resources are same as given in the above example except Persistent class Employee.java where you don't need to use any annotation and hibernate.cfg.xml file where you need to specify mapping resource of the hbm file.

The hbm file should be like this:

```
<?xml version='1.0' encoding='UTF-8'?>  
<!DOCTYPE hibernate-mapping PUBLIC  
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"  
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">  
  
<hibernate-mapping>  
    <class name="com.javatpoint.Employee" table="em">  
        <id name="id">  
            <generator class="native"></generator>  
  
        </id>  
        <property name="name"></property>  
        <property name="job"></property>  
        <property name="salary"></property>  
    </class>  
  
    <query name="findEmployeeByName">  
        <![CDATA[from Employee e where e.name = :name]]>  
    </query>
```

</hibernate-mapping>

Q. the two locking strategies in Hibernate?

Ans: Optimistic: In Optimistic locking we assume that multiple transactions can complete without affecting each other. So we let the transactions do their work without locking the resources initially. Just before the commit, we check if any of the resource has changed by another transaction, then we throw exception and rollback the transaction. Pessimistic: In Pessimistic locking we assume that concurrent transactions will conflict while working with same resources. So a transaction has to first obtain lock on the resources it wants to update. The other transaction can proceed with same resource only after the lock has been released by previous transaction.

Q. What inheritance mapping strategies are supported by Hibernate?

Ans: Hibernate supports following inheritance mapping strategies between classes and tables:
Table per class hierarchy: In case of multiple types of books, we can have one book class and one book table. We can store all child classes of book like- HardcoverBook, PaperbackBook etc in same table book. But we can identify the subclasses by a BookType column in Book table.
Table per subclass: In this case we can have separate table for each kind of book. HardcoverBook table for HardcoverBook book class. PaperbackBook table for PaperbackBook book class. And there will be a parent table, Book for Book class.
Table per concrete class: In this case also we have separate table for each kind of book. But in this case we have even inherited properties defined inside each table. There is no parent table Book for Book class, since it is not a concrete class.

Q. What is the difference between merge and update ?

Ans: Use update() if you are sure that the session does not contain an already persistent instance with the same identifier, and merge() if you want to merge your modifications at any time without consideration of the state of the session. In other words, update() is usually the first method you would call in a fresh session, ensuring that reattachment of your detached instances is the first operation that is executed.

Q. How do you store an image in Oracle using Hibernate ?

Ans: We can use byte[] type field in json. It absorbs base64 encoded string of image, which is directly saved and retrieved from db by Hibernate easily.

Or

1. Create a table with column having data type as blob
2. In the HBM file, specify the type as binary

Q. Explain the relationships in Hibernate.

Ans:

1. One to One
2. One to Many
3. Many to One
4. Many to Many

5. One to Many / Many to One bi-directional

Q. What is the use of cascade in hbm file?

Ans:1) cascade="none", the default, tells Hibernate to ignore the association.

2) cascade="save-update" tells Hibernate to navigate the association when the transaction is committed and when an object is passed to save() or update() and save newly instantiated transient instances and persist changes to detached instances.

3) cascade="delete" tells Hibernate to navigate the association and delete persistent instances when an object is passed to delete().

4) cascade="all" means to cascade both save-update and delete, as well as calls to evict and lock.

5) cascade="all-delete-orphan" means the same as cascade="all" but, in addition, Hibernate deletes any persistent entity instance that has been removed (dereferenced) from the association (for example, from a collection).

6) cascade="delete-orphan" Hibernate will delete any persistent entity instance that has been removed (dereferenced) from the association (for example, from a collection).

Q. What are web services ?

Ans:Web services are client and server applications that communicate over the World Wide Web's (WWW) HyperText Transfer Protocol (HTTP). Web services provide a standard means of inter operating between software applications running on a variety of platforms and frameworks.

Main characteristics of the Web Services are :

1. Interoperability
2. Extensibility
3. Machine processable descriptions.

for example in simple words , when we call somebody so the person dialing and calling is the client application , while person receiving the call is server applicationand "hello" word is the protocol as similar to HTTP request

Q. What is the difference between SOA and a web service?

Ans:There are some key differences between Web services and SOA. Web services define a web technology that can be used to build applications that can send /receive messages using SOAP over HTTP. However, SOA is an architectural model for implementing loosely coupled service based applications. Web services can be used to implement SOA applications. Even though web service approach to SOA has

become very popular, it is only a single method of implementing SOA. SOA can be implemented using any other service-based technology (e.g. CORBA and REST).

Q. What is SOAP?

Ans: SOAP (Simple Object Access Protocol) is a transport protocol for sending and receiving requests and responses on XML format, which can be used on top of transport protocols such as HTTP, SMTP, UDP, etc

Q. What is REST?

Ans: REST (REpresentational State Transfer) is an architectural style by which data can be transmitted over transport protocol such as HTTP(S).

Q. What is the difference between a REST web service and a SOAP web service?

Ans: Below are the main differences between REST and SOAP web service

REST supports different formats like text, JSON and XML; SOAP only supports XML;

REST works only over HTTP(S) on a transport layer; SOAP can be used different protocols on a transport layer;

REST works with resources, each unique URL is some representation of a resource; SOAP works with operations, which implement some business logic through different interfaces;

SOAP based reads can't be cached, for SOAP need to provide caching; REST based reads can be cached;

SOAP supports SSL security and WS-security(Web Service-security); REST only supports SSL security;

SOAP supports ACID (Atomicity, Consistency, Isolation, Durability); REST supports transactions, but it is neither ACID compliant nor can provide two phase commit.

Q. How to decide which one of web service to use REST or SOAP?

Ans: "REST vs SOAP" we can rephrase to "Simplicity vs Standard". Of course, "Simplicity" with REST at most cases wins, it wins in performance, scalability and support for multiple data formats, but SOAP is favored where service requires comprehensive support for security (WS-security) and transactional safety (ACID).

Q. What is meant by JAX-WS and JAX-RS?

Ans:

JAX-WS stands for java api for xml based web services and has the specification and api for soap based web services.

JAX-RS stands for java API for xml based Rest Services and has the specification and api to implement RESTful web services.

Q. What is WSDL?

Ans: WSDL (Web Services Description Language) is an XML format for describing web services and how to access them.

Q. What is JAXB?

Ans:JAXB (Java Architecture for XML Binding) is a Java standard that defines how Java objects are converted from and to XML. It makes reading and writing of XML via Java relatively easy.

Q. What is MTOM?

Ans:MTOM (Message Transmission Optimization Mechanism) is a mechanism for transmitting large binary attachments with SOAP messages as raw bytes, allowing for smaller messages.

Q. What is XOP?

Ans:XOP (XML-binary Optimized Packaging) is a mechanism defined for the serialization of XML Information Sets that contain binary data, as well as deserialization back into the XML Information Set.

Q. What is a SOAP envelope element?

Ans:SOAP envelop element is the root element of a SOAP message which defines the XML document as a SOAP message.

An example:

```
<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">

...
  Message information
...
</soap:Envelope>
```

Q. What is the SOAP encoding?

Ans:SOAP encoding is a method for structuring the request which is suggested within the SOAP specification, known as the SOAP serialization.

Q. What does SOAP encodingStyle attribute defines?

Ans:SOAP encodingStyle defines the serialization rules used in a SOAP message. This attribute may appear on any element, and is scoped to that element's contents and all child elements not themselves containing such an attribute. There is no default encoding defined for a SOAP message.

An example:

SOAP-ENV:encodingStyle="http://www.w3.org/2001/12/soap-encoding"

Q. How to call WS after all the stub/skeletons are created

Ans:

Steps are :

1. Create the instance of the Service Class by passing the WSDL URL to the constructor.
2. Create the porttype by invoking the get method on the Service created in step one
3. Call the web service methods by passing in the appropriate parameters

Q. How to consume a web service given a WSDL. Explain all the tools required to create the stubs/skeleton, configuration etc.

Ans:

Steps:

1. Generate the stubs from the wsdl using wsdl2java
2. Create an instance of the porttype by giving it the wsdl url
3. Invoke the method on the object.

Q. What are the steps to implement user name token profile?

Ans:

Steps:

1. Implement the call back handler class that can return the password for the given user id.
2. Configure the User Name token profile security in the cxf-servlet.xml

Q. What are the steps to configure Basic Authentication?

Ans:Two Steps:

- 1) Configure the user in tomcat-users.xml or other application servers configuration
- 2) Configure the basic authentication using the login-config and auth-constratints elements in the web.xml of our JEE Web Application.

Q. What are the steps to implement java first web service?

Ans:Four Steps:

1. Create the Web Services Interface , Implementation and other bean Classes
2. Annotate the web service interface and implementation with the

3. Annotate the beans with JAXB annotations
4. Define the Endpoint in the cxf-servlet.xml (When CXF Is Used)

Q. List the steps to exchange files using MTOM?

Ans:Two Steps:

1. Use the DataHandler type as the bean property or parameters to the web service method.
2. Enable the MTOM property on the endpoint in the cxf-servlet.xml

Q. Explain the structure of a WSDL File?

Ans:Abstract Section:

WSDL:types – Has the xml schema that can be used to build the messages

WSDL:messages: the request and response xml messages built using the schema in the types section above.

WSDL:Operation: Each operation that the web service exposes.

WSDL:PortType: Groups the operations or web service methods together.

Physical Portion:

WSDL:Binding – We define the binding for all the operations which controls how the soap message is generated.

WSDL:service: The location/url of the service using which it can be accessed.

Q. What are 2 styles web service's endpoint by using JAX-WS?

Ans:RPC (remote procedure call) style web service in JAX-WS;

document style web service in JAX-WS.

Q. What is encoding rules for header entries?

Ans:a header entry is identified by its fully qualified element name, which consists of the namespace URI and the local name. All immediate child elements of the SOAP Header element must be namespace-qualified.

the SOAP encodingStyle attribute may be used to indicate the encoding style used for the header entries.

the SOAP mustUnderstand attribute and SOAP actor attribute may be used to indicate how to process the entry and by whom.

Q. What is the wsimport tool?

Ans:The wsimport tool is used to parse an existing Web Services Description Language (WSDL) file and generate required files (JAX-WS portable artifacts) for web service client to access the published web services.

Q. What is the ws-gen tool?

Ans: The ws-gen tool is used to parse an existing web service implementation class and generates required files (JAX-WS portable artifacts) for web service deployment.

Q. What is a SOAP Engine?

Ans: SOAP Engine does two things:

Dispatches the SOAP Requests to the appropriate endpoints

Serializes the soap messages to language objects and De-Serializes the language objects to soap messages.

Q. What WS Standards are you aware of?

Ans: WS-Security

MTOM

WS-Addressing

WS-Policy

WS-Reliable Messaging

Q. What types of operations are available in WSDL?

Ans: There are four operations available:

1. One-way, where the operation can receive a message but will not return a response.
2. Request-response, where the operation can receive a request and will return a response.
3. Solicit-response, where the operation can send a request and will wait for a response.
4. Notification, where the operation can send a message but will not wait for a response

RESTful

Q. What are the steps to design a REST Web Service?

Ans: Four Steps:

Identify the objects/resources using the class diagram. There are the nouns in the object model.

Create the URIs to access these resources.

Choose the HTTP methods to bind to the operations on the resources.

Identify the data format that needs to be supported and come up with sample requests and responses

Q. Between HTTP POST and PUT requests?

Ans: The HTTP methods POST and PUT aren't the HTTP equivalent of the CRUD's create and update. They both serve a different purpose. It's quite possible, valid and even preferred in some occasions, to use POST to create resources, or use PUT to update resources.

Use PUT when you can update a resource completely through a specific resource.

If you do not know the actual resource location, for instance, when you add a new object, but do not have any idea where to store it, you can POST it to an URL, and let the server decide the actual URL.

As soon as you know the new resource location, you can use PUT again to do updates to the blue stapler article. But as said before: you CAN add new resources through PUT as well. The next example is perfectly valid if your API provides this functionality

PUT and POST are both unsafe methods. However, PUT is idempotent, while POST is not

Q. What kind of output formats can one generate using RESTful web service?

Ans:Xml,JSON,TEXT ETC .

Q. What are HTTP methods supported by REST?

Ans:

GET;

POST;

PUT;

DELETE;

OPTIONS;

HEAD.

Q. Whether can use GET request instead of POST to create a resource?

Ans:It is not possibly, because GET can't change a resource.

Q. What are frameworks available to implement REST web services?

Ans:Jersey, Restlet, EasyRest, etc.

Q. What is the difference between AJAX and REST?

Ans:In Ajax, the request are sent to the server by using XMLHttpRequest objects; REST have a URL structure and a request/response pattern the revolve around the use of resources;

Ajax eliminates the interaction between the customer and server asynchronously; REST requires the interaction between the customer and server;

Ajax is a set of technology; REST is a type of software architecture and a method for users to request data or information from servers.

Q. What does a @Path annotation do?

Ans:@Path annotation binds URI pattern to a Java method.

Q. What does a @PathParam do?

Ans:@PathParam annotation injects the value of URI parameter that defined in @Path expression.

Q. What does a @MatrixParam do?

Ans:@MatrixParam are a set of "name=value" in URI path.

Q. How will you handle application error scenarios in RESTful web service?

Ans: There are two types of errors.

1) Standard Errors

2) Application Errors

Standard Errors Can be handled in three ways:

We can returns the error code using the JAX-RS Response Objects Methods

We can throw a WebApplicationException and set the http error status on it.

We can throw a specific exception.

Application Errors or exceptions can be handled using one of the three ways above or my writing JAX-RS ExceptionMappers. There exception mappers should be marked with the @Provider JAX-RS annotations and should be configured in the cxf-servlet.xml.

Q. Can a RESTful web service generate output in various formats based on some parameter received from the client?

Ans:Yes. Based on the accept http header the RESTful provider will know what MIME TYPE data the client can accept send out the appropriate response back.

Q. SOAP WS or REST?

Ans:

REST based Web service is preferred due to its simplicity, performance, scalability, and support for multiple data formats. SOAP is favored where service requires comprehensive support for security and transactional reliability.

The answer really depends on the functional and non-functional requirements. Does the service expose data or business logic? (REST is a better choice for exposing data, SOAP WS might be a better choice for logic).

SOAP is more verbose. Because of the SOAP XML Elements (SOAP – Envelope ,Header, Body) overhead SOAP needs more bandwidth.

Thanks!!

Best of luck for your future endeavor!!